Reversible Radix-4 Booth Multiplier for DSP Applications

Nagamani A N, Nikhil R, Manish Nagaraj
Dept. of Electronics and Communications
PES Institute of Technology
Bangalore, 560085, India
Email: nagamani@pes.edu, nikhilkashyap.95@gmail.com, manishnagaraj1@gmail.com

Vinod Kumar Agrawal
Dept. of Information and Sciences
Director - CORI
PES Institute of Technology
Bangalore, 560085, India
Email: vk.agrawal@pes.edu

***Abstract*—Power dissipation has become the major concern for circuit design and implementation. Reversible Logic is the best alternative to Irreversible Logic in terms of low power consumption. Circuits designed using reversible logic have a wide array of applications. The Quantum Cost, Garbage Outputs, Ancillary Inputs and Delay are some of the parameters of reversible circuits that can be used to determine their efficiency and compare them with existing works. Optimization of these parameters are highly essential. Garbage Outputs is an important parameter that must be considered. This paper presents a design for a Reversible Radix-4 Booth Multiplier that is optimized in Garbage Cost and Ancillary inputs. The design proposed is capable of both signed and unsigned multiplication. The optimization in Garbage Cost ensures lower heat dissipation. The Encoded Booth Algorithm or Radix-4 Booth Algorithm reduces the number of partial products generated in signed multiplication to half the number generated using a Radix-2 signed multiplier making it suitable for Digital Signal Processors. The design proposed is compared to existing multiplier circuits and the parameters are tabulated.**

I. Introduction

Conventional computational circuits and VLSI systems use irreversible gates which have very high power dissipation owing to rapid switching of internal signals. As per Landauer's principle [1], there is an energy loss due to loss in data. There is a loss of $kTln2$ joules for every bit of data that is lost, where k is the Boltzmann constant ($1.3806 \times 10^{-23} m^2 kg s^{-2} K^{-1}$) and T is the absolute temperature at which the computation occurs.

Loss of data is an inherent property of logic gates in use at present. Let us consider the two input $NAND$ gate. It has two input lines (say A and B) and one output line (say Y). The output Y is given by $\overline{A \cdot B}$. We can see that there is no unique one-to-one mapping between the inputs and output. Furthermore, the input signals are lost altogether. Thus, the basic logic gates (like $AND, OR, NAND, NOR$) are "irreversible".

In every instance of these gates, there will be data loss and thus, a small energy loss as well. But, as the complexity of a circuit increases, it's gate count increases, accumulating to a huge energy loss. The more the number of logic elements packed into smaller volumes at higher clock frequencies, the more is the power dissipated. This creates no less than three problems: Increase in monetary expenditure, quicker battery drains in portable systems and excessive overheating of the system [2].

Reversible Logic is the solution to this problem [3]. There is a one-to-one mapping between the inputs and the outputs of reversible logic and hence the inputs can be regenerated. Energy can thus be recycled. This avoids data loss and hence, there is no energy loss. Fredkin and Toffoli proved that minimizing garbage outputs, ancillary inputs increases the efficiency of the reversible logic circuit [4]. The reduction of garbage output lines is of utmost importance in reducing the heat loss of a reversible circuit [5], [6], [7]. In [5], it has been stated that when faced with the question of reduction in gate count or garbage output, the preference must be given to the minimization of garbage output. Reversible logic has applications in quantum computing, optical computing etcetera.

Multiplication forms the heart of any Digital Signal Processor (DSP). Hence, the power dissipation, speed and area of a computer system depends on the multiplier used within. Since the ever improving technology and increasing complexity of circuitry urges for optimization in area and delay, the need for a multiplier occupying lesser area and having lesser delay is of paramount importance. One such multiplier architecture was invented by Andrew Booth [8] in 1950. The underlying principle incorporated in this algorithm is that shifting bits is much quicker than adding them.

In this paper, a design for an Radix-4 Booth Multiplier is proposed using synthesizable Reversible logic gates. This design optimises the Garbage output count and the Ancillary input count with respect to the design proposed in [9]. The family of Reversible gates and the metrics governing them are studied in section II. The Radix-4 Booth Algorithm is discussed in section III. The proposed design for the Reversible Radix-4 Booth Multiplier is presented in IV. It's implementation and an example is undertaken in section V.

II. Preliminaries

A. Basics Of Reversible Logic

1) Reversible Gates: A reversible gate is an n-input, n-output circuit that maintains a unique mapping between the

978-1-5090-1746-1/16/$31.00 © 2016 IEEE

inputs and the outputs in a circuit. They reduce bit loss and dissipation of heat and they are adaptable for quantum computing. In the circuits involving reversible gates, fan-out is strictly one and feedback is prohibited.

2) Constant Inputs/Ancillary Inputs: In order to achieve a required output function, inputs which are kept constant at all time instances are used.

3) Garbage Outputs: Garbage outputs refer to the number of unwanted outputs that are added to make a circuit reversible. They are neither used as inputs to subsequent units nor are they primary outputs.

4) Quantum Cost: The total number of 1×1 and 2×2 quantum gates that make up a reversible gate is known as its Quantum Cost. The Quantum cost of all 1×1 and 2×2 gates are taken to be 1.

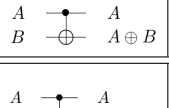
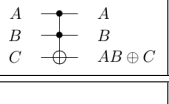
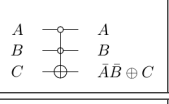
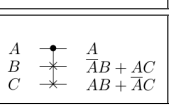
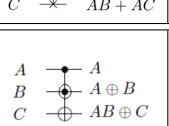
5) Delay: The number of gates between the input and the output of a particular line of the reversible circuit is referred to as the Delay of the circuit. The definition is based on two assumptions: (i) Computation of each gate is performed in one unit time and (ii) all inputs to the circuit are available before the computation begins.

In this paper, we have used the logical depth as measure of the delay proposed by Mohammadi and Eshghi [10]. The Quantum cost of a reversible gate is taken as it's delay.

B. Family of Reversible Gates

Table I illustrates the various Reversible gates used in this paper with their input lines, output lines, circuit diagram and quantum cost.

TABLE I: Family of Reversible Gates.

Gate	Circuit Diagram	Quantum Cost
CNOT/Feynman $(A, B) \rightarrow (A, A \oplus B)$		1
Toffoli $(A, B, C) \rightarrow (A, B, AB \oplus C)$		5
Negative controlled Toffoli $(A, B, C) \rightarrow (A, B, \bar{A}\bar{B} \oplus C)$		6
Fredkin $(A, B, C) \rightarrow (A, \bar{A}B + AC, AB + \bar{A}C)$		5
Peres $(A, B, C) \rightarrow (A, A \oplus B, AB \oplus C)$		4

C. Literature Survey

The existing literature research consists of many multipliers that have been proposed optimizing various parameters, to perform multiplication for various applications. The multiplier design proposed in [11] uses a binary tree based design and optimizes the garbage cost as well as ancillary inputs of the circuit. Garbage free reversible ripple carry adders based on Bennet scheme are used to add the partial products generated reducing the garbage cost. However, the drawback of the design is the high quantum cost.

Reversible multipliers proposed in [12] are based on the Urdhva Tiryakbhayam algorithm. The design optimizes garbage cost, ancillary inputs as well as Total Reversible Logic Implementation cost (TRLIC). However, due to the arrangement of the adders in the design, the accuracy of the design is low and a multiplier design with a correction algorithm is proposed in [13] to overcome this effect. Using multi-control input Toffoli synthesis a multiplier has been proposed in [14]. Garbage outputs and ancillary inputs are optimized for unsigned multipliers in [15].

Many researchers have also proposed designs for performing signed multiplication of numbers. Two such design are proposed in [16]. Modified Baugh-Wooley algorithm have been implemented in both the designs. The Booth's algorithm for signed multiplication is proposed in [8]. This provides an efficient way of calculating the product of signed numbers. The design in [9] is based on this algorithm. Various other authors have proposed new gates to make for simpler and faster circuits in unsigned multipliers.

The main inspiration behind this work is to design an Radix-4 Booth Multiplier using only synthesizable Reversible gates (CNOT, Toffoli, Peres, Fredkin).

III. Radix-4 Booth Algorithm

There are various techniques and architectures to perform multiplication in DSPs. The factors differentiating these architectures from one another are delay, throughput, power dissipation, area, complexity of the design. Basic multiplier architectures first compute all the partial products and then use various adders, adder trees to find the sum of the partial products.

Radix-2 array multipliers compute partial products taking into account one bit of the multiplier at a time. Radix-4 Booth multiplier takes into account three bits of the multiplier at a time. This reduces the number of partial products from N to $N/2$. Let us consider two 4-bit signed numbers $X = X_3X_2X_1X_0$ and $Y = Y_3Y_2Y_1Y_0$ with the MSB indicating the sign (1 for negative and 0 for positive). Three bits of the multiplier X are grouped at a time and encoded [17]. The encoding map is shown in table II. The multiplicand Y is multiplied with $\sum$, where $\sum = X_{i-1} + X_i - 2 \times X_{i+1}$ is the encoded output. $-Y$ is taken as the two's complement of Y, while $-2Y$ is the two's complement of the left shifted Y. $2Y$ is simply Y shifted one position to the left. The sign extension bits are the MSB of each row of partial products. The sign extension bits are 0 for positive and 1 for negative partial products. The next row of partial products are then written after shifting it left not once as in the conventional multipliers, but twice as it has four times the weight. The rows of partial products are added to give the final product.

TABLE II: Arithmetic operations to be performed

X_{i+1}	X_i	X_{i-1}	$\sum$	A	M	S	Operation
0	0	0	0	0	0	0	No Action
0	0	1	1Y	1	0	0	Add
0	1	0	1Y	1	0	0	Add
0	1	1	2Y	1	1	0	Shift left and Add
1	0	0	-2Y	1	1	1	Shift left and Subtract
1	0	1	-1Y	1	0	1	Subtract
1	1	0	-1Y	1	0	1	Subtract
1	1	1	-0	0	0	1	No Action

Three control signals are generated from $\sum$. The first indicating if there should be any action on the multiplicand array (Y) and is denoted by A. A is 0 when there is no action and 1 when there is to be an arithmetic operation. The second decides if the multiplicand array (Y) should be shifted left ($2Y$ is same as shifting Y one time to the left) and it is denoted by M. M is 1 when $\sum$ is ± 2 and 0 otherwise. The third decides whether the multiplicand array should be added to subtracted from the sum and it is denoted by S. S is 0 for addition and 1 for subtraction. The circuit design is proposed in the next section.

IV. Proposed Design

In this paper, three units namely the Control Unit, the Arithmetic Unit and Multiplexer Units have been designed to achieve signed multiplication using Radix-4 Booth Algorithm.

A. Control Unit (CU)

The partial products need to be only shifted when $X_{i+1}X_iX_{i-1}$ bits are either 000 or 111. In other cases, an arithmetic operation must be performed first and then the partial products must undergo a shift. The partial products need to be shifted an additional time if $X_{i+1}X_iX_{i-1}$ bits are 011 or 100 as per table II. This is controlled using the three signals A, S and M generated from the designed CU.

$$S = X_{i+1} \tag{1}$$

$$\overline{A} = \overline{X_{i+1}} \cdot \overline{X_i} \cdot \overline{X_{i-1}} + X_{i+1} \cdot X_i \cdot X_{i-1} \tag{2}$$

$$M = \overline{X_{i+1}} \cdot X_i \cdot X_{i-1} + X_{i+1} \cdot \overline{X_i} \cdot \overline{X_{i-1}} \tag{3}$$

The circuit implementation of the control unit is as shown in the figure 1.

B. Arithmetic Unit (AU)

When the bits $X_{i+1}X_iX_{i-1}$ are not 000 or 111 the partial products need to be arithmetically manipulated. The arithmetic operation to be performed based on the bits are shown in table II. Hence, the AU needs to act as an adder-subtracter unit based on the signals S and A generated in the CU. When the AU acts as an adder, the sum and carry of the three inputs a (current partial product), b (new line of partial product), c (carry-in bit) are given by,

$$Sum = a \oplus b \oplus c,\ C_{out} = a \cdot b \oplus b \cdot c \oplus a \cdot c \tag{4}$$

When the AU acts a subtracter, the difference and the borrow of a, b, c are given by,

$$Sum = a \oplus b \oplus c,\ C_{out} = \bar{a} \cdot b \oplus b \cdot c \oplus \bar{a} \cdot c \tag{5}$$

Hence the adder-subtracter unit equations can be generalised as,

$$Sum = a \oplus (b \oplus c) \cdot A,\ C_{out} = (a \oplus S) \cdot (b \oplus c) \oplus b \cdot c, \tag{6}$$

Where S and A are the control signals from CU. The circuit implementation of AU is as shown in the figure 2.

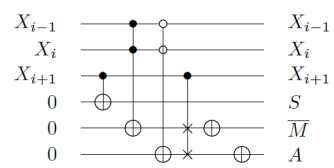


Figure 1: CU circuit.

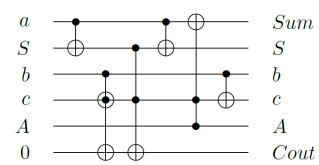


Figure 2: AU circuit.

C. Multiplexer unit (MUX)

When the bits $X_{i+1}X_iX_{i-1}$ are either 011 or 100 then the partial products need to be shifted an additional time. This is achieved in the design using a MUX circuit. A 9:8 MUX is shown in figure 4 with the control signal $\overline{M}$ from the CU. The outputs are extracted as $R_7R_6.....R_1R_0 = ssssY_3Y_2Y_1Y_0$ when $M = 0$ and $R_7R_6.....R_1R_0 = sssY_3Y_2Y_1Y_00$ when $M = 1$.

A subsequent row of AUs require the same 'b' inputs. A reverse MUX circuit is used to retrieve back the inputs from the previous stage. It's circuit consists of the mirror image of the MUX circuit in figure 4. Although the $[i+1:i]$ MUX has one garbage output, it will be used later in the reverse $[i+1:i]$ MUX. Thus every MUX and the reverse MUX pair will have three garbage outputs. The last MUX will have two garbage outputs. The parameters of AU, CU and $[i+1:i]$ MUX are shown in table III.

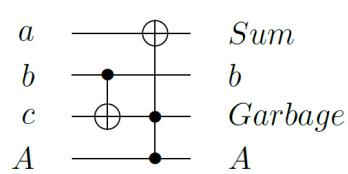


Figure 3: Modified structure for the left most AU in each stage of the architecture in figure 8.

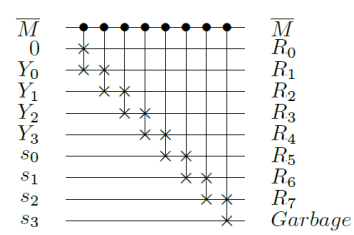


Figure 4: 9:8 MUX

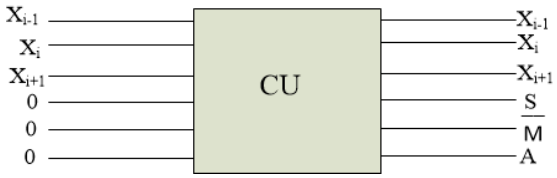


Figure 5: CU block diagram.

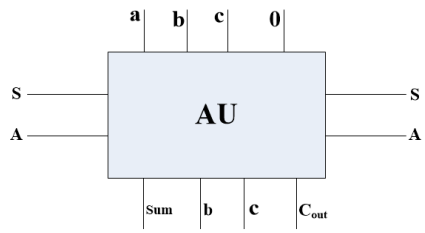


Figure 6: AU block diagram.

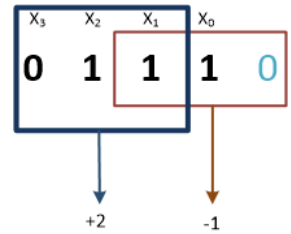


Figure 7: Multiplier bit groupings.

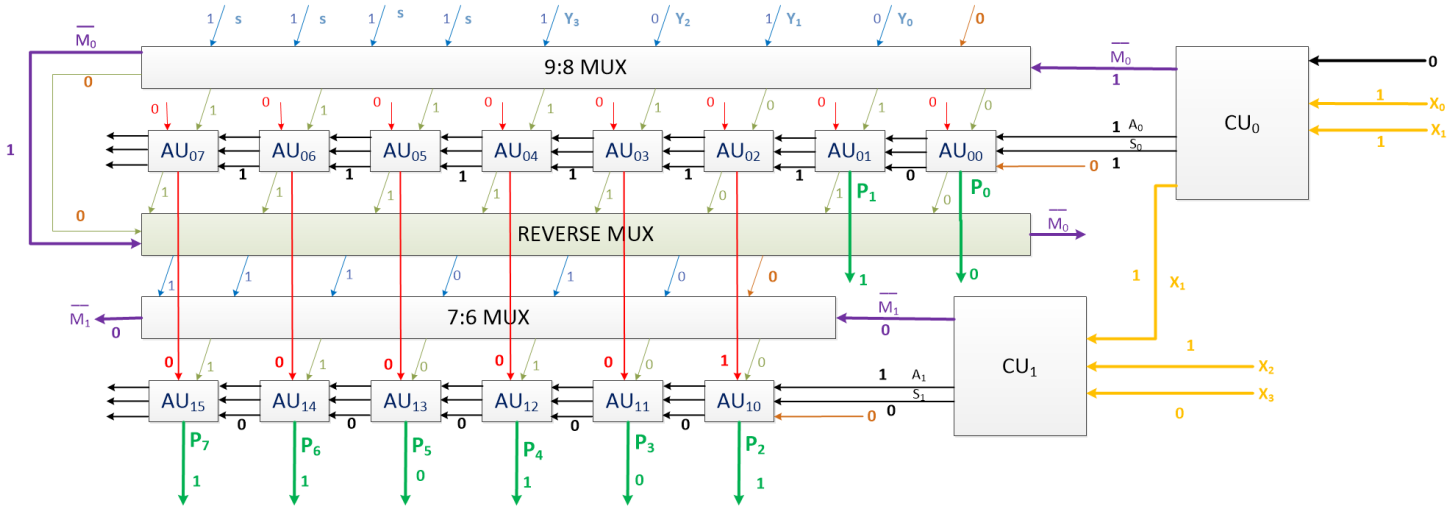


Figure 8: An example of 4×4 multiplication with $X = 0111_2(+7_{10})$, $Y = 1010_2(-6_{10})$ and $P = 11010110_2(-42_{10})$.

TABLE III: Parameters for AU and SU units

Parameter	AU	CU	$[i+1:i]$ MUX
Quantum Cost	17	19	$5i$
Garbage outputs	0	0	0
Ancillary Inputs	1	3	1
Gate Count	6	6	i
Delay	16△	19△	$5i$△
Logic depth	5	4	i

V. Implementation Of Radix-4 Booth's Algorithm

The multiplier (X) and the multiplicand (Y) are (taken in two's complement if they are negative) fed to the Radix-4 Booth multiplier. The architecture used to implement the algorithm for 4×4 multiplication is shown in figure 8. The multiplier $X_3X_2X_1X_0$ is appended with an implicit 0 to the right of the LSB. The bits are then grouped in threes and fed to the control units as shown in figure 8. The multiplicand $Y_3Y_2Y_1Y_0$ bits are fed to the first four AU blocks and the sign bit s (here, s is the MSB of Y) is fed to the remaining AU blocks of the first row. The left most AU block of each row is shown in figure 3.

The first three bits $X_1, X_0, 0$ are grouped and fed to the first control unit CU_0. The signal $\overline{M_0}$ generated by CU_0 is fed to the 9:8 MUX. The MUX shifts the input numbers depending on M_0. Based on the control signals A_0 and S_0 generated by CU_0, the array of AUs in the first row will add or subtract the multiplicand to or from 0. The two LSB bits of the resulting product is extracted as P_1P_0 and the remaining bits are passed as inputs 'a' to the next array of AUs. The Y bits along with the sign bits are regenerated in the same order using a reverse MUX and fed to the next MUX.

The next two bits X_3X_2 along with the regenerated X_1 are grouped and fed to the next control unit CU_1. The signal $\overline{M_1}$ generated here controls the second level MUX. The partial products are then added or subtracted based on the second set of CU controls A_1 and S_1. The final SUM generated denotes the product ($P_7P_6P_5P_4P_3P_2P_1P_0$) of the two numbers X and Y. Table VI shows the parameters of each of the stages of the 4×4 multiplier proposed.

Let us consider an example, with $X = 7_{10}$ and $Y = -6_{10}$. They can be written as $X = 0111_2$ and $Y = 1010_2$ (Two's complement of 6_{10} or 0110_2 is 1010_2). The bits of X are grouped as shown in figure 7.

Thus, the multiplicand Y has to be multiplied by +2 -1. We now trace the working of the architecture shown in figure 8 for the above example. Figure 8 shows the working of the Radix-4 Booth Multiplier. The product evaluated is 11010110_2. As the MSB is 1, the number is negative. It's two's complement is 00101010_2 (42_{10}). Thus, the product is -42_{10}.

The design is generalized for $n \times n$ multipliers as follows.

TABLE IV: Parameters for 4×4 Signed Multiplier.

Stage	Quantum Cost	Garbage Output	Ancillary Input
First level [9:8] MUX	40	0	1
Control Unit CU_0	19	0	4
First Row of AUs	125	3	16
Control Unit CU_1	19	0	3
First level Reverse [9:8] MUX	40	3	0
Second Level [7:6] MUX	30	2	0
Second Row of AUs	91	3	6
Total	364	11	30

An $n \times n$ multiplier has $n/2$ stages. The first $(n/2)-1$ stages has one CU block, a $[2n-2(k-1)+1 : 2n-2(k-1)]$ MUX, a row consisting of $2n-2(k-1)$ AU blocks, and a reverse MUX of the same dimensions as that of the MUX, where k is the stage number. The $(n/2)^{th}$, i.e, the last stage has a CU block, a $[n+3 : n+2]$ MUX, and a row consisting of $n+2$ AU blocks. The leftmost AU in each stage has the structure shown in figure 3. The parameters for each block are shown in table III. Based on these values, the generalised $n \times n$ multiplier will have parameters as shown in table V.

TABLE V: Parameters for $n \times n$ Signed Multiplier.

Stage	Quantum Cost	Garbage Output	Ancillary Input
AU Blocks	$(51n^2 + 12n)/4$	$3n/2$	$(3n^2+12n)/4$
CU Blocks	$19n/2$	0	$3n/2$
MUX Blocks	$(15n^2-20)/2$	$(3n-2)/2$	1
Total	$(81n^2+50n-40)/4$	$(3n-1)$	$(3n^2+16n+8)/4$

VI. Comparison with Existing Work

The proposed multiplier design has an improvement in garbage cost as well as ancillary inputs as compared to the designs in [9] and the first proposed design in [16]. Table VI shows the comparison of the parameters of the designs for a 4×4 signed multiplier proposed in this paper and [9] and 5×5 signed multiplier proposed in [16]. The multiplier designs in [13], [14] and [15] are for unsigned numbers.

TABLE VI: Comparison of Multipliers.

Parameter	Proposed Design	Design in [9]	Design in [16]*
Quantum Cost	364	315	231
Ancillary Inputs	30	63	56
Garbage Cost	11	67	56

* It must be noted that the design in [16] is a 5×5 multiplier.

VII. Conclusion

This paper presents a Reversible Radix-4 Booth Multiplier. It has been optimized for minimal Garbage outputs and Ancillary inputs. This design may be used for both signed and unsigned multiplication. Furthermore, this design reduces the number of partial product by half when compared to existing Radix-2 design. Thus, reducing the area of the circuit by almost half. With respect to the designs in [9] and [16], the garbage output count has been reduced by 83.6% and 80.4% while the ancillary input count has been reduced by 52.4% and 46.4% respectively. This design has reduced heat dissipation and occupies less area and thus can be integrated with Quantum Computing ALUs. Future work is to further optimise Quantum cost and reduce delay.

References

[1] R. Landauer, "Irreversibility and heat generation in the computing process," *IBM journal of research and development*, vol. 5, no. 3, pp. 183–191, 1961.

[2] R. C. Merkle. Reversible Logic. [Online]. Available: http://www.zyvex.com/nanotech/reversible.html

[3] C. H. Bennett, "Logical reversibility of computation," *IBM journal of Research and Development*, vol. 17, no. 6, pp. 525–532, 1973.

[4] E. Fredkin and T. Toffoli, *Conservative logic*. Springer, 2002.

[5] D. Maslov and G. W. Dueck, "Garbage in reversible design of multiple output functions," in *6th International Symposium on Representations and Methodology of Future Computing Technologies*, 2003, pp. 162–170.

[6] W. N. Hung, X. Song, G. Yang, J. Yang, and M. Perkowski, "Optimal synthesis of multiple output boolean functions using a set of quantum gates by symbolic reachability analysis," *Computer-Aided Design of Integrated Circuits and Systems, IEEE Transactions on*, vol. 25, no. 9, pp. 1652–1663, 2006.

[7] M. Haghparast, M. Mohammadi, K. Navi, and M. Eshghi, "Optimized reversible multiplier circuit," *Journal of Circuits, Systems, and Computers*, vol. 18, no. 02, pp. 311–323, 2009.

[8] A. D. Booth, "A signed binary multiplication technique," *The Quarterly Journal of Mechanics and Applied Mathematics*, vol. 4, no. 2, pp. 236–240, 1951.

[9] J. Sultana, S. K. Mitra, and A. R. Chowdhury, "On the Analysis of Reversible Booth's Multiplier," in *VLSI Design (VLSID), 2015 28th International Conference on*. IEEE, 2015, pp. 170–175.

[10] M. Mohammadi and M. Eshghi, "On figures of merit in reversible and quantum logic designs," *Quantum Information Processing*, vol. 8, no. 4, pp. 297–318, 2009.

[11] S. Kotiyal, H. Thapliyal, and N. Ranganathan, "Circuit for reversible quantum multiplier based on binary tree optimizing ancilla and garbage bits," in *VLSI Design and 2014 13th International Conference on Embedded Systems, 2014 27th International Conference on*. IEEE, 2014, pp. 545–550.

[12] R. Saligram and T. Rakshith, "Optimized reversible vedic multipliers for high speed low power operations," in *Information & Communication Technologies (ICT), 2013 IEEE Conference on*. IEEE, 2013, pp. 809–814.

[13] A. Das and M. J. K. Bha, "Design optimization of vedic multiplier using reversible logic," in *International Journal of Engineering Research and Technology*, vol. 3, no. 3 (March-2014). ESRSA Publications, 2014.

[14] H. Bhagyalakshmi and M. Venkatesha, "Optimized multiplier using reversible multi-control input toffoli gates," *International Journal of VLSI Design & Communication Systems*, vol. 3, no. 6, p. 27, 2012.

[15] H. Jayashree, H. Thapliyal, H. R. Arabnia, and V. Agrawal, "Ancilla-input and garbage-output optimized design of a reversible quantum integer multiplier," *The Journal of Supercomputing*, pp. 1–17, 2016.

[16] E. P. A. Akbar, M. Haghparast, and K. Navi, "Novel design of a fast reversible wallace sign multiplier circuit in nanotechnology," *Microelectronics Journal*, vol. 42, no. 8, pp. 973–981, 2011.

[17] W. N. HE *et al.*, *Cmos Vlsi Design: A Circuits And Systems Perspective, 3/E*. Pearson Education India, 2006, ch. Datapath Subsystems.

[18] D. M. Miller, D. Maslov, and G. W. Dueck, "A transformation based algorithm for reversible logic synthesis," in *Design Automation Conference, 2003. Proceedings*. IEEE, 2003, pp. 318–323.

[19] M. Arabzadeh and M. Saeedi. (2013) RC Viewer+, version 2.5. [Online]. Available: http://ceit.aut.ac.ir/QDA/RCV.htm